



Lab Sheet-02

Data Structure I (BTCS209P)

B.Tech III Semester (Session 2026-2027)

Objective:

To understand and implement fundamental sorting algorithms using C++, and to analyze their working, algorithms, time complexity, and space complexity.

Requirements:

C++ Compiler / IDE

VS Code, Code::Blocks, Dev-C++, or any suitable C++ environment

Basic knowledge of arrays, loops, functions, and conditional statements

Concepts Covered:

1. Bubble Sort
2. Insertion Sort
3. Selection Sort
4. Merge Sort
5. Quick Sort

1. Sorting

Sorting is the process of arranging data elements in a particular order, usually ascending or descending. In this lab, five important sorting algorithms are implemented and analyzed.

2. Comparison-Based Sorting

Bubble Sort, Insertion Sort, Selection Sort, Merge Sort, and Quick Sort are comparison-based sorting algorithms. They determine the relative order of elements by comparing them.

1. Bubble Sort

Bubble Sort repeatedly compares adjacent elements and swaps them if they are in the wrong order. After every pass, the largest unsorted element moves to its correct position.

Algorithm:

BUBBLE_SORT(A, n)

1. Repeat for $i = 0$ to $n - 2$
2. Set swapped = false
3. Repeat for $j = 0$ to $n - i - 2$
4. If $A[j] > A[j + 1]$
5. Swap $A[j]$ and $A[j + 1]$
6. Set swapped = true
7. If swapped == false
8. Break
9. Stop



Time and Space Complexity:

Best: $O(n)$, Average: $O(n^2)$, Worst: $O(n^2)$, Space: $O(1)$

Properties:

In-place and stable. The optimized version can stop early when no swap occurs.

2. Insertion Sort

Insertion Sort builds the sorted portion of the array one element at a time. Each new element (key) is compared with elements in the sorted portion and inserted into its correct position.

Algorithm:

INSERTION_SORT(A, n)

1. For $i = 1$ to $n - 1$
2. $key = A[i]$
3. $j = i - 1$
4. While $j \geq 0$ and $A[j] > key$
5. $A[j + 1] = A[j]$
6. $j = j - 1$
7. $A[j + 1] = key$
8. Stop

Time and Space Complexity:

Best: $O(n)$, Average: $O(n^2)$, Worst: $O(n^2)$, Space: $O(1)$

Properties:

In-place, stable, and adaptive. It performs well for small or nearly sorted arrays.

3. Selection Sort

Selection Sort repeatedly finds the minimum element from the unsorted part and places it at the beginning of that part.

Algorithm:

SELECTION_SORT(A, n)

1. For $i = 0$ to $n - 2$
2. $minIndex = i$
3. For $j = i + 1$ to $n - 1$
4. If $A[j] < A[minIndex]$
5. $minIndex = j$
6. Swap $A[i]$ and $A[minIndex]$
7. Stop

Time and Space Complexity:

Best: $O(n^2)$, Average: $O(n^2)$, Worst: $O(n^2)$, Space: $O(1)$

Properties:

In-place. Standard Selection Sort is not stable.



4. Merge Sort

Merge Sort uses the Divide and Conquer technique. It divides the array into smaller subarrays, recursively sorts them, and then merges the sorted subarrays.

Algorithm:

MERGE_SORT(A, left, right)

1. If $\text{left} < \text{right}$
2. $\text{mid} = (\text{left} + \text{right}) / 2$
3. MERGE_SORT(A, left, mid)
4. MERGE_SORT(A, mid + 1, right)
5. MERGE(A, left, mid, right)
6. Stop

MERGE(A, left, mid, right)

1. Create temporary arrays for the left and right parts
2. Compare the front elements of both parts
3. Copy the smaller element into the merged array
4. Copy any remaining elements
5. Copy the merged elements back into A
6. Stop

Time and Space Complexity:

Best: $O(n \log n)$, Average: $O(n \log n)$, Worst: $O(n \log n)$, Space: $O(n)$

Properties:

Stable in its standard implementation. Uses additional memory for merging.

5. Quick Sort

Quick Sort uses the Divide and Conquer technique. It selects a pivot, partitions the array around the pivot, and recursively sorts the left and right subarrays.

Algorithm:

QUICK_SORT(A, low, high)

1. If $\text{low} < \text{high}$
2. $\text{pivotIndex} = \text{PARTITION}(A, \text{low}, \text{high})$
3. QUICK_SORT(A, low, $\text{pivotIndex} - 1$)
4. QUICK_SORT(A, $\text{pivotIndex} + 1$, high)
5. Stop

PARTITION(A, low, high)

1. $\text{pivot} = A[\text{high}]$
2. $i = \text{low} - 1$
3. For $j = \text{low}$ to $\text{high} - 1$
4. If $A[j] < \text{pivot}$
5. $i = i + 1$
6. Swap $A[i]$ and $A[j]$
7. Swap $A[i + 1]$ and $A[\text{high}]$
8. Return $i + 1$



Time and Space Complexity:

Best: $O(n \log n)$, Average: $O(n \log n)$, Worst: $O(n^2)$, Space: $O(\log n)$ average recursion stack

Properties:

In-place in the usual array implementation. Standard Quick Sort is not stable.

Lab Exercises

1. Write a C++ program to implement Bubble Sort in ascending order. Display the array before sorting and after each pass.
2. Write a C++ program to implement Bubble Sort in descending order and display the final sorted array.
3. Write a C++ program to implement Insertion Sort in ascending order. Display the array after each insertion/pass.
4. Write a C++ program to implement Selection Sort in ascending order. Display the minimum element selected in each pass.
5. Write a C++ program to implement Merge Sort using separate functions for Merge Sort and Merge operations.
6. Write a C++ program to implement Quick Sort using the last element as the pivot.
7. Write a C++ program to implement Quick Sort and display the array after each partition.
8. Write a C++ program to implement all five sorting algorithms: Bubble, Insertion, Selection, Merge, and Quick Sort.
9. Take the same input array and sort it using Bubble Sort, Insertion Sort, and Selection Sort. Compare the number of comparisons and swaps.
10. Take the same input array and sort it using Merge Sort and Quick Sort. Compare their time and space complexity.
11. Write a C++ program that allows the user to select any one of the five sorting algorithms using a menu-driven approach.
12. Write a C++ program to sort an array in ascending order and then in descending order using a sorting algorithm of your choice.
13. Write a C++ program to count the number of comparisons performed by Bubble Sort, Insertion Sort, and Selection Sort for a given input.
14. Write a C++ program to demonstrate the best-case behavior of Bubble Sort and Insertion Sort using an already sorted array.
15. Write a C++ program to test Quick Sort on sorted, reverse-sorted, and randomly arranged arrays and observe the effect of pivot selection.



Practice / Analysis Questions

1. Why is Bubble Sort called Bubble Sort?
2. What is the role of the key element in Insertion Sort?
3. Why does Selection Sort perform $O(n^2)$ comparisons even in the best case?
4. Why is Merge Sort called a Divide and Conquer algorithm?
5. What is the purpose of the Merge operation in Merge Sort?
6. What is a pivot in Quick Sort?
7. Why can Quick Sort have $O(n^2)$ worst-case time complexity?
8. Which sorting algorithms are stable among the five algorithms used in this lab?
9. Which sorting algorithms are in-place?
10. Which sorting algorithm requires $O(n)$ additional space in its standard implementation?
11. Compare Merge Sort and Quick Sort in terms of best, average, and worst-case time complexity.
12. Compare Bubble Sort, Insertion Sort, and Selection Sort in terms of their basic working principle.

Instructions & Guidelines

Write a separate C++ program for each assigned exercise. Use functions wherever appropriate to make the programs modular and easy to understand. Do not use the built-in `sort()` function for implementing these sorting algorithms. Test each program with different inputs, including sorted, reverse-sorted, and random arrays. Display the input and output clearly.

Add comments to important parts of the program.

Mention the time and space complexity of each implemented algorithm.

For comparison-based exercises, use the same input array for a fair comparison.

Submission

Submit the C++ source code files for the assigned tasks along with a brief report containing the algorithm, sample input/output, and time and space complexity analysis of the implemented sorting algorithms.